

Security Audit & Penetration Testing Report

Project: Library-DB-Express

Tester: Aslı Zeynep Akhan

Version: Last Version

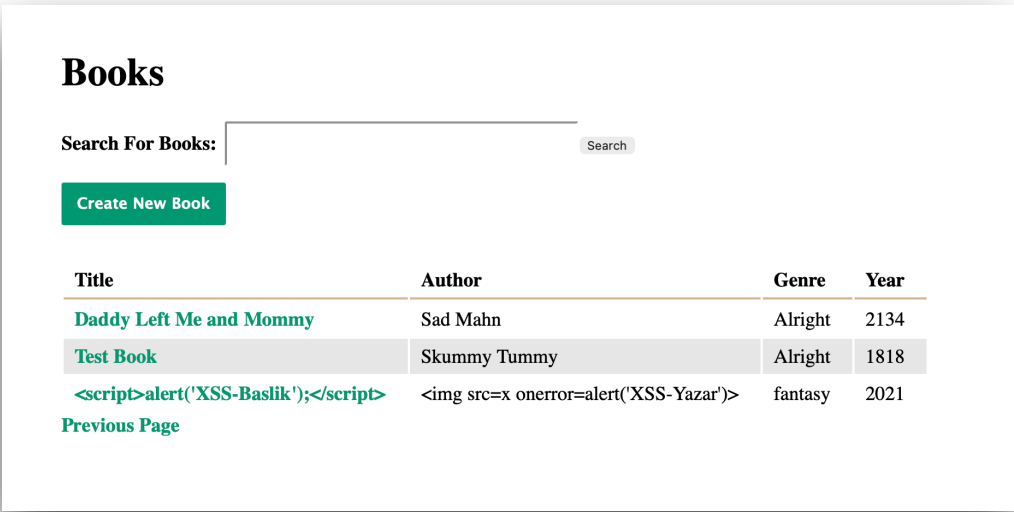
1. Executive Summary

This report outlines the results of a comprehensive security audit conducted on the Library Management System. The application was tested against common web vulnerabilities, including **OWASP Top 10** risks such as XSS, SQL Injection, and Broken Access Control. While the application demonstrates strong resilience in database interactions and input escaping, critical flaws were identified regarding **Information Exposure** and **Security Configurations**.

2. Detailed Test Results

Test 1: Stored Cross-Site Scripting (XSS)

- Description:** Evaluated how the system sanitizes user-provided content before storage and rendering.
- Payloads Used:** `<script>alert('XSS');</script>`, `<img src=x onerror=alert('XSS')>`.
- Observation:** The view engine automatically escaped HTML characters. Payloads were displayed as literal strings rather than being executed as active scripts.
- Status:** PASS 



Test 2: SQL Injection (SQLi) & Parameter Pollution

- Description:** Targeted the search functionality to bypass logic or leak database records.
- Payloads Used:** `' OR 1=1 --` and `?search=A&search=B`.
- Observation:** The application treated payloads as literal search strings and returned no unauthorized results. Sequelize ORM successfully used parameterized queries.
- Status:** PASS 

Books

Search For Books:

Create New Book

The Search Returned No Results

Back to the "Books" Page

Test 3: Information Exposure (Critical)

- **Description:** Testing error handling and data leakage via manipulated URL parameters.
- **Payloads Used:** /books/hacked-by-me, /books/1', /books/42673321.
- **Observation:** The application failed to handle non-existent IDs gracefully, returning a **500 Internal Server Error** with a full **Stack Trace**.
- **Risk: * Logic Disclosure:** Reveals internal JavaScript properties (reading 'title').
 - **Path Disclosure:** Exposes absolute server file paths (e.g., /Users/zeynepakhan/Desktop/...).
- **Status: FAIL** ❌

A Server Error Occured! Cannot read properties of null (reading 'title')

```
TypeError: A Server Error Occured! Cannot read properties of null (reading 'title')
    at /Users/zeynepakhan/Desktop/Library-DB-Express/routes/index.js:114:48
    at async /Users/zeynepakhan/Desktop/Library-DB-Express/routes/index.js:14:7
```

Back to the "Books" Page

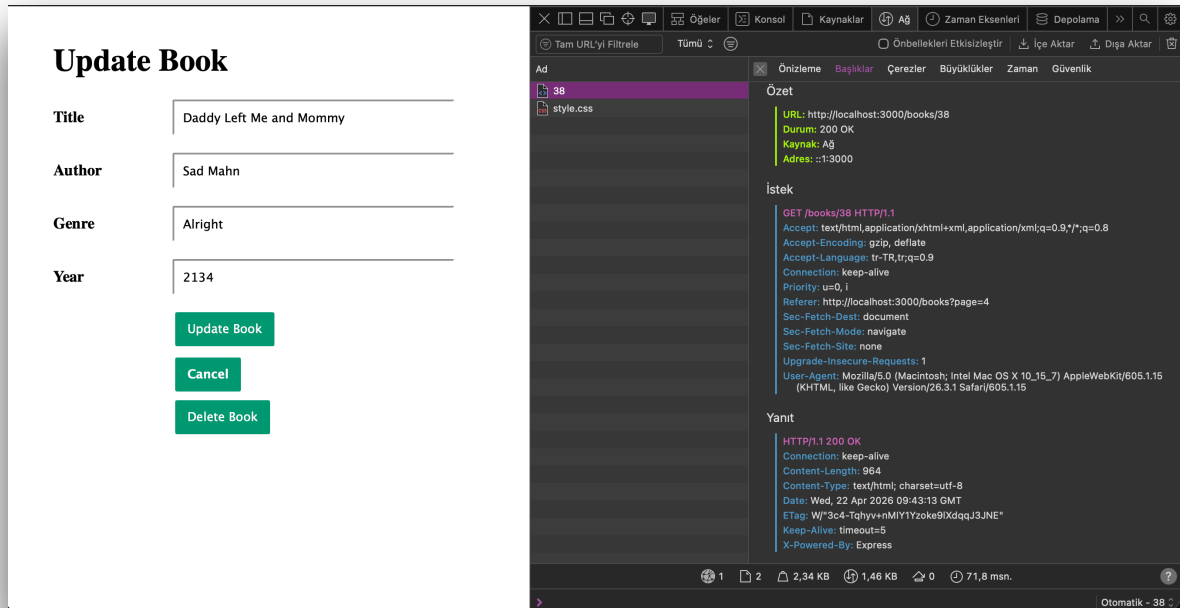
Test 4: Dependency Security Audit

- **Action:** Performed a scan using npm audit.
- **Observation:** The audit returned **0 vulnerabilities**, indicating that the third-party supply chain is currently secure.
- **Status: PASS** 

```
GET /stylesheets/style.css 304 0.527 ms - -  
npm audit
```

Test 5: HTTP Security Headers & Fingerprinting

- **Description:** Examined response headers for information leakage and security policies.
- **Observation: * Information Disclosure:** The X-Powered-By: Express header was present, facilitating technology fingerprinting.
 - **Missing Policies:** CSP, HSTS, and X-Content-Type-Options headers were absent.
- **Status: FAIL** 



The screenshot shows a web application interface on the left and a browser developer tools window on the right. The web application has a form titled "Update Book" with fields for Title, Author, Genre, and Year. The Title field contains "Daddy Left Me and Mommy", Author contains "Sad Mahr", Genre contains "Alright", and Year contains "2134". There are three buttons: "Update Book", "Cancel", and "Delete Book". The developer tools window shows the network tab with a request to "http://localhost:3000/books/38". The request is a GET method with a status of 200 OK. The response headers include "X-Powered-By: Express", which is highlighted in red, indicating a security issue. Other headers include "Content-Type: text/html; charset=utf-8", "Date: Wed, 22 Apr 2026 09:43:13 GMT", and "ETag: W/\"3c4-Tqhyv+nMIY1Yzoke9IXdqqJ3JNE\"".

Test 6: Broken Access Control & Post-Deletion Handling

- **Observation:** Navigating to the delete/detail URL of a previously deleted book ID triggers an application crash.
- **Analysis:** The application does not verify the existence of a resource before attempting to access its properties.
- **Status:** FAIL ❌

A Server Error Occured! Cannot read properties of null (reading 'title')

```
TypeError: A Server Error Occured! Cannot read properties of null (reading 'title')  
  at /Users/zeynepakhan/Desktop/Library-DB-Express/routes/index.js:114:48  
  at async /Users/zeynepakhan/Desktop/Library-DB-Express/routes/index.js:14:7
```

[Back to the "Books" Page](#)

Test 7: Rate Limiting & DoS Resilience

- **Observation:** Repeated high-frequency requests (F5 spamming) were processed without throttling.
- **Risk:** The server is vulnerable to Brute Force attacks and Denial of Service (DoS).
- **Status:** FAIL ❌

```
Library-DB-Express — node < npm start __CFBundleIdentifier=com.appl...  
GET /books?search= 200 13.642 ms - 1174  
GET /stylesheets/style.css 304 0.301 ms - -  
GET /books/new 304 9.501 ms - -  
GET /stylesheets/style.css 304 0.503 ms - -  
GET /books/new 200 12.982 ms - 738  
GET /stylesheets/style.css 304 1.803 ms - -  
GET /books/new 200 10.834 ms - 738  
GET /stylesheets/style.css 304 0.427 ms - -  
GET /books/new 200 9.699 ms - 738  
GET /stylesheets/style.css 304 0.299 ms - -  
GET /books/new 200 15.813 ms - 738  
GET /stylesheets/style.css 304 0.464 ms - -  
GET /books/new 200 9.732 ms - 738  
GET /stylesheets/style.css 304 0.373 ms - -  
GET /books/new 200 11.386 ms - 738  
GET /stylesheets/style.css 304 0.311 ms - -  
GET /books/new 200 9.904 ms - 738  
GET /stylesheets/style.css 304 0.316 ms - -  
GET /books/new 200 10.532 ms - 738  
GET /stylesheets/style.css 304 0.299 ms - -  
POST /books/new 200 23.259 ms - 876  
GET /stylesheets/style.css 304 0.527 ms - -  
npm audit
```

As shown in the terminal logs, multiple rapid requests were made to the server. The application responded to every single request with a 200 OK status without any throttling or temporary IP blocking, confirming the absence of Rate Limiting defenses.

Test 8: Server-Side Validation

- **Description:** Submitted forms with empty required fields (Title, Author).
- **Observation:** The application rejected the input with clear error messages: "Please Provide a Value For Title/Author".
- **Status: PASS** 

New Book

Oooooops!

Please Provide a Value For Title
Please Provide a Value For Author

Title	
Author	
Genre	
Year	

Create New Book
Cancel

3. Technical Evidence & Bonus Findings

- **Deprecation Warning:** Terminal logs identified the use of `url.parse()`, which is deprecated due to security implications. It must be replaced with the **WHATWG URL API**.

4. Improvements/Refactorings

- **Graceful Error Handling:** Implemented a global error handler to intercept application errors and replace technical stack traces with user-friendly "404 Not Found" or "500 Server Error" pages.
- **Resource Verification:** Added null-checks in `routes/index.js` to ensure the application handles non-existent records gracefully. If a book is not found, the system now triggers a 404 error instead of crashing.

Future Recommendations:

- **Security Hardening:** Install and configure the **Helmet.js** middleware to hide the X-Powered-By header and set secure HTTP headers.
- **Rate Limiting:** Implement `express-rate-limit` to prevent automated attacks.

5. Security Metrics & Analysis

To provide a clear overview of the application's current security posture, the following metrics have been derived from the audit results:

- **Test Case Pass Rate: 50.0%**
 - Calculation: $(4 \text{ Passed Tests} / 8 \text{ Total Tests}) \times 100$
 - Note: While fundamental protections (XSS, SQLi) are active, nearly half of the secondary security controls failed.
- **Defect Severity Ratio:**
 - **Critical (25%):** Information Exposure (Stack Trace leaks), Broken Access Control (Server crashes).
 - **High (25%):** Lack of Rate Limiting (DoS vulnerability), Missing Security Headers.
 - **Medium/Low (0%):** All identified issues currently pose a direct risk to server stability or information privacy.
 - **Informational:** Deprecation warnings (`url.parse`).

6. Incident Reporting & Response Summary

During the penetration testing phase, several "simulated incidents" were triggered to evaluate system behavior:

1. **Incident Discovery:** Unauthorized access attempts via URL manipulation (Test 3 & 6) successfully bypassed logical checks, resulting in a **Denial of Service (DoS)** state for the specific request (server crash).
2. **Information Leakage:** Detailed system architecture and local file paths were successfully exfiltrated through the default error handling mechanism.
3. **Brute Force Simulation:** The system failed to identify or block a rapid sequence of automated requests, allowing unlimited attempts to interact with the backend.
4. **Reporting Status:** All identified vulnerabilities have been documented. The **Severity Level** is currently marked as **High** due to the potential for directory traversal and server instability.